

CODEIT CareerOS

AI-Powered Career Readiness & Skill Gap Platform

Product Tagline

Test. Analyze. Improve. Become Industry Ready.

Product Vision

CODEIT CareerOS is a career-readiness ecosystem for engineering and technology students.

The platform helps a student answer: "Where am I today, what career am I targeting, what skills am I missing, and exactly what should I do next?"

CareerOS is not only an LMS, course platform, job portal or resume builder. It is a career diagnostic + personalized action platform connecting:

Assessment → Skill Gap → AI Guidance → Learning → Projects → OJT → Internship → Resume → Interview → Recruitment

1. PRIMARY OBJECTIVES

The system must allow a student to:

- 1. Create a career profile.**
- 2. Select a target career.**
- 3. Take a role-specific skill assessment.**
- 4. Receive skill-wise scores.**
- 5. Identify career gaps.**
- 6. Upload and analyze a CV.**
- 7. Analyze LinkedIn profile information.**
- 8. Receive a personalized career roadmap.**
- 9. Get recommended projects.**
- 10. Track progress.**
- 11. Interact with an AI Career Assistant.**
- 12. Generate a Career Readiness Report.**
- 13. Eventually connect with CODEIT OJT, internships and recruitment.**

The architecture must support future College/TPO portal, Employer portal, OJT management, Internship management, Recruitment, Job matching, Corporate programs and Multi-college/multi-tenant support.

2. MVP SCOPE

Phase 1 – Core CareerOS

Student:

- Registration/Login
- Student profile
- Education details
- Career selection
- Skill assessment
- Assessment result
- Career gap analysis
- Career readiness score
- Career roadmap
- CV upload
- CV analysis
- LinkedIn profile analysis
- Project recommendations
- Progress tracking
- AI Career Assistant
- Career Readiness Report PDF

Admin:

- Admin login
- Student management
- Career management
- Skill management
- Skill-category management
- Question management
- Assessment management
- Competency framework management
- Roadmap management
- Project management
- AI configuration
- Report access
- Dashboard/analytics

Do not start with employer portal, job matching or complicated college features. Keep them architecturally possible but implement them later.

3. HIGH-LEVEL USER JOURNEY

Student Registration → Create Career Profile → Select Target Career → Career Assessment → Assessment Result → Career Gap Analysis → Career Readiness Score → Personalized Roadmap → AI Career Assistant → Recommended Projects → CV Analysis → LinkedIn Analysis → Progress Tracking → Career Readiness Report → OJT / Internship / Interview → Recruitment

4. TECHNOLOGY STACK

Backend:

- Java 21 LTS
- Spring Boot 3.x
- Spring Web
- Spring Security
- Spring Data JPA
- Hibernate
- Bean Validation
- Maven
- REST API
- OpenAPI / Swagger

Database:

PostgreSQL. Use relational database design. Do not use MongoDB for the core CareerOS data in MVP.

5. FRONTEND

Recommended:

- React
- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui

Frontend communicates with Java backend through REST APIs.

Architecture: Next.js / React → REST API → Spring Boot → PostgreSQL

6. CLOUD / INFRASTRUCTURE

Use AWS.

Recommended architecture:

CloudFront → Frontend → Application Load Balancer → AWS ECS / Fargate → Spring Boot Application → AWS RDS PostgreSQL

Additional services:

- AWS S3 – CV/document storage
- AWS SES – email
- AWS CloudWatch – monitoring/logging
- AWS Secrets Manager – secrets
- AWS IAM – access control
- AWS Route 53 – domain/DNS
- AWS Certificate Manager – SSL

7. DEVELOPMENT TOOLS

IDE: IntelliJ IDEA or Eclipse / Spring Tool Suite

Version Control: Git, GitHub

Repositories: codeit-careeros-backend, codeit-careeros-frontend

API Testing: Postman, Swagger UI

Database: PostgreSQL, pgAdmin, DBeaver optional

Build: Maven

Code Quality: SonarQube optional, Checkstyle optional, JUnit, Mockito

8. BACKEND PROJECT STRUCTURE

```
com.codeit.careeros
├── config
├── controller
├── service
├── repository
├── entity
├── dto
├── mapper
├── security
├── exception
├── validation
├── ai
├── └── controller
├── └── service
├── └── tools
├── └── prompt
├── assessment
├── career
├── cv
├── linkedin
├── roadmap
├── project
├── report
├── progress
├── common
```

Use clean separation between Controller, Service, Repository, Entity and DTO. Do not expose JPA entities directly through APIs.

9. AUTHENTICATION

Roles:

STUDENT

ADMIN

COLLEGE_ADMIN — Future

EMPLOYER — Future

MENTOR — Future

SUPER_ADMIN — Future

Use Spring Security and JWT authentication. Passwords must use BCrypt hashing. Never store plain-text passwords.

10. STUDENT PROFILE

Registration fields:

Student ID, Full Name, Email, Mobile, Password, Date of Birth (optional), College, Degree, Branch, Graduation Year, Semester, Location, GitHub URL, LinkedIn URL, Portfolio URL, Profile Photo.

Skills:

Programming Languages, Frameworks, Databases, Cloud, DevOps, Data, AI/ML, Cybersecurity, Testing, Soft Skills.

Allow students to select skills, but do not treat self-declared skills as verified competency.

11. CAREER MASTER

Career must be configurable by Admin.

Example careers:

Java Developer, Python Developer, Full Stack Developer, MERN Stack Developer, Cloud Engineer, DevOps Engineer, AWS Cloud Engineer, Data Analyst, Data Scientist, AI/ML Engineer, Cybersecurity Analyst, QA Engineer, Business Analyst, HR Executive, Business Development Executive.

Each career should have:

Career ID, Career Name, Description, Category, Difficulty Level, Required Skills, Skill Weight, Recommended Projects, Roadmap, Interview Topics.

12. COMPETENCY FRAMEWORK

Each career should contain required skills.

Example: DevOps Engineer

Linux 15%

Networking 10%

Git 10%

AWS 15%

Docker 15%

Jenkins / CI-CD 10%

Kubernetes 10%

Terraform 10%

Monitoring 5%

Security Basics 5%

Total 100%

The administrator must be able to modify these weights. Do not hard-code career skills in Java.

13. SKILL LEVELS

Standardized levels:

BEGINNER, BASIC, INTERMEDIATE, ADVANCED, EXPERT

Assessment output:

STRONG, GOOD, DEVELOPING, NEEDS_IMPROVEMENT, PRIORITY_GAP

14. SKILL ASSESSMENT

Student selects target career. System loads the relevant assessment.

Question types:

- MCQ
- Multiple Select — future
- Scenario Based — future
- Coding Question — future
- Practical Assessment — future

MVP should start with MCQs.

Each question contains:

Question ID, Career ID, Skill ID, Question, Option A, Option B, Option C, Option D, Correct Answer, Difficulty, Marks, Explanation, Status.

15. ASSESSMENT ENGINE

Assessment should calculate:

Total Questions, Attempted, Correct, Incorrect, Score, Percentage, Skill-wise Score.

Example:

Overall Score: 72%

Linux 85%

Git 80%

AWS 68%

Docker 55%

Kubernetes 42%

Terraform 35%

16. CAREER GAP ENGINE

Compare Required competency VS Student demonstrated competency.

Example:

Required Docker level: 80

Student Docker score: 55

Gap = 25

Classification:

0–10 Strong

11–20 Good

21–35 Developing

36–50 Needs Improvement

51+ Priority Gap

Thresholds must be configurable.

17. CAREER READINESS SCORE

Create a CODEIT-specific indicator: Career Readiness Score.

Example weights:

Skill Assessment 30%

Skill Coverage 20%

Projects 15%

CV Alignment 15%

LinkedIn 5%

Interview Readiness 15%

Total 100%

The score is a CODEIT Career Readiness Indicator. It must NOT be presented as a job guarantee, placement probability, hiring prediction or employer ranking. Admin must be able to configure weights.

18. DASHBOARD

Student dashboard should show:

Welcome, Student

Target Career

Career Readiness

Skill Assessment

CV Alignment

LinkedIn

Projects

Interview Readiness

Top Career Actions:

1. Improve Docker

2. Learn Kubernetes

3. Build AWS project

4. Improve CV project descriptions

5. Complete mock interview

Use charts:

- Skill radar
- Progress bars

- Readiness score
- Gap chart
- Roadmap timeline

19. CAREER ROADMAP

Roadmap should be generated from career + gaps.

Example: DevOps 90-Day Roadmap

Days 1–30: Linux, Networking, Git, AWS fundamentals

Days 31–60: Docker, Jenkins, CI/CD, AWS deployment

Days 61–90: Kubernetes, Terraform, Monitoring, Capstone Project, Mock Interview

Roadmap must be configurable by Admin.

20. CV UPLOAD

Supported: PDF, DOC, DOCX

Storage: AWS S3

Database stores: File URL, File Name, File Type, Upload Date, Student ID, Version

Do not store large files directly in PostgreSQL.

21. CV PARSER

PDF: Apache PDFBox

DOCX: Apache POI

Extract:

Name, Email, Phone, Education, Skills, Experience, Projects, Certifications, Achievements, Links.

Then send structured content to AI analysis.

22. CV ANALYSIS

Analyze:

Profile completeness, Career alignment, Skills, Projects, Experience, Education, Achievements, Keywords, Formatting, Project descriptions, Missing sections, Generic statements.

Example:

Target Role: Java Developer

CV Analysis: Skills Alignment 72%

Missing/Weak Areas: Spring Boot, REST API, Unit Testing

AI must never invent experience, projects, skills, certifications or achievements.

23. LINKEDIN ANALYSIS

Do NOT scrape LinkedIn.

Student provides:

- LinkedIn URL
- Or manually entered profile information
- Or future approved integration

Analyze:

Headline, About, Education, Experience, Skills, Projects, Certifications, Featured, Profile photo, Contact information, Portfolio links.

Output:

Profile Completeness: 72%

Improve: Headline, About section, Project descriptions, Skills, Featured section.

24. PROJECT RECOMMENDATION ENGINE

Projects should be mapped to:

Career, Skill, Difficulty, Technology, Expected duration, Learning outcome.

Example:

Student: Data Analyst

Strong: Excel, SQL

Gap: Power BI, Data Visualization

Recommended: Sales Analytics Dashboard

Technology: Excel, SQL, Power BI

Difficulty: Intermediate

Duration: 2 weeks

25. PROGRESS TRACKING

Track:

Assessment completed, Skill completed, Roadmap step completed, Project completed, CV updated, LinkedIn updated, Mock interview completed.

Example:

Career Readiness 64% → 71% → 78%

Maintain history rather than overwriting previous results.

26. AI CAREER ASSISTANT

Name: CODEIT Career AI Assistant

Purpose: Provide personalized career guidance using verified CareerOS data.

27. AI TECHNOLOGY

Recommended:

- Spring AI
- LLM API

- Tool Calling
- Structured Outputs

AI provider should be configurable. Do not hard-code the entire application to one model provider.

Architecture:

React / Next.js → Spring Boot → Career AI Service → Spring AI → LLM Provider → Tool Calling → CareerOS APIs → PostgreSQL

28. AI AGENT TOOLS

The AI agent should NOT directly access PostgreSQL.

Create controlled tools/APIs:

getStudentProfile()

getTargetCareer()

getSkillAssessment()

getSkillGaps()

getCareerReadiness()

getCVAnalysis()

getLinkedInAnalysis()

getCareerRoadmap()

getRecommendedProjects()

getProgress()

getInterviewReadiness()

Future:

createDailyTask()

startMockInterview()

evaluateMockInterview()

generateProjectPlan()

generateCareerReport()

Do NOT implement multiple autonomous agents in MVP. Start with one controlled AI Career Assistant.

29. AI EXAMPLE

Student asks: "What should I learn next?"

AI should:

1. Get student profile

2. Get target career

3. Get assessment

4. Get skill gaps

5. Get roadmap

6. Identify highest-priority gap

7. Explain why

8. Give next practical action

Example response:

Your current target is DevOps Engineer.

Your highest-priority gaps are Kubernetes, Terraform and Docker.

Your next recommended focus is Docker.

Why? Your assessment shows a lower demonstrated competency in Docker compared with other DevOps skills.

Next action: Build a Dockerized Java application and deploy it locally.

The AI should explain recommendations based on CareerOS data.

30. AI SAFETY / DATA RULES

The AI must:

- Never fabricate student information.
- Never fabricate experience.
- Never invent certifications.
- Never invent projects.
- Never claim a student is guaranteed a job.
- Never expose another student's data.
- Never access database directly.
- Only use authorized tools.
- Respect student permissions.
- Clearly distinguish student-provided information from AI suggestions.

AI-generated content should be stored separately from verified student data.

31. AI CHAT HISTORY

Store:

Conversation ID, Student ID, User Message, AI Response, Timestamp, Model, Token Usage — optional.

Do not store sensitive information unnecessarily. Provide option to delete conversation history.

32. DATABASE – CORE TABLES

Minimum tables:

users

roles

student_profiles

careers

skills

career_skills
skill_categories
assessment_tests
assessment_questions
question_options
assessment_attempts
assessment_answers
skill_scores
career_gaps
career_readiness_scores
roadmaps
roadmap_items
projects
project_skills
student_projects
cv_documents
cv_analysis
linkedin_profiles
linkedin_analysis
progress_tracking
ai_conversations
ai_messages
reports
audit_logs

Future:

colleges, college_students, employers, employer_users, internships, ojt_programs, jobs, applications, recruitment

33. IMPORTANT DATABASE PRINCIPLE

Do not hard-code:

- Careers
- Skills
- Questions
- Skill weights
- Roadmap
- Project mappings
- Readiness weights

These should be database-driven. Admin must be able to modify them.

34. API STRUCTURE

Base: /api/v1

Authentication:

POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout

Student:

GET /students/me
PUT /students/me
GET /students/me/dashboard

Careers:

GET /careers
GET /careers/{id}
GET /careers/{id}/skills

Assessment:

GET /assessments/{careerId}
POST /assessments/{id}/start
POST /assessments/{id}/submit
GET /assessments/attempts/{id}

Career Gap:

GET /career-gaps/me
POST /career-gaps/recalculate

Roadmap:

GET /roadmaps/me
PUT /roadmaps/{id}/items/{itemId}

Projects:

GET /projects/recommended
GET /projects/{id}
POST /projects/{id}/start
POST /projects/{id}/complete

CV:

POST /cv/upload
GET /cv
GET /cv/{id}/analysis
POST /cv/{id}/analyze

LinkedIn:

POST /linkedin/profile
GET /linkedin/profile
POST /linkedin/analyze

Progress:

GET /progress/me
POST /progress

Reports:

```
POST /reports/career-readiness/generate
GET /reports
GET /reports/{id}
```

AI:

```
POST /ai/chat
GET /ai/conversations
GET /ai/conversations/{id}
DELETE /ai/conversations/{id}
```

35. ADMIN APIs

```
/admin/students
/admin/careers
/admin/skills
/admin/career-skills
/admin/assessments
/admin/questions
/admin/roadmaps
/admin/projects
/admin/reports
/admin/analytics
/admin/ai/config
```

36. ADMIN DASHBOARD

Show:

Total Students, Active Students, Assessments Completed, Average Readiness, Top Career Choices, Top Skill Gaps, CV Analysis Count, LinkedIn Analysis Count, Projects Completed, AI Usage.

Career distribution:

DevOps, Java, Python, Data Analyst, Full Stack, AI/ML, Cybersecurity, etc.

37. CAREER READINESS REPORT

Generate downloadable PDF.

Suggested pages:

- 1 Student Profile
- 2 Target Career
- 3 Assessment Results
- 4 Skill Gap Analysis
- 5 Career Readiness Score
- 6 CV Analysis
- 7 LinkedIn Analysis
- 8 Project Recommendations
- 9 Career Roadmap

10 30/60/90 Day Action Plan

Report should contain:

CODEIT CareerOS

Career Readiness Report

Student Name

Target Career

Generated Date

Career Readiness Indicator

38. FILE STORAGE

AWS S3 structure:

careeros/

students/

{studentId}/

profile/

cv/

reports/

Use private S3 buckets. Generate temporary signed URLs when necessary. Do not expose public CV URLs.

39. SECURITY REQUIREMENTS

Mandatory:

- HTTPS
- JWT
- BCrypt
- Role-based authorization
- Input validation
- File type validation
- File size limits
- SQL injection protection through JPA/parameterized queries
- XSS protection
- CORS configuration
- Rate limiting for AI APIs
- Secure secrets
- Audit logging
- S3 private bucket

Never commit API keys, JWT secrets, AWS keys or database passwords to GitHub. Use AWS Secrets Manager and environment variables.

40. FILE UPLOAD SECURITY

Allow only PDF, DOC, DOCX.

Recommended maximum: 5–10 MB.

Validate MIME type, extension and file size. Malware scanning in future.

Rename uploaded files using UUID. Never trust original filename.

41. LOGGING

Use SLF4J and Logback.

Log:

Request ID, User ID, API, Status, Execution time, Errors.

Never log:

Passwords, JWT tokens, API keys, CV full content, sensitive personal data.

42. ERROR HANDLING

Create global exception handler.

Standard response:

```
{
  "success": false,
  "message": "Assessment not found",
  "errorCode": "ASSESSMENT_NOT_FOUND",
  "timestamp": "2026-09-17T12:00:00"
}
```

Use appropriate HTTP status codes.

43. API DOCUMENTATION

Use OpenAPI / Swagger.

Every API must have:

Description, Request, Response, Error response, Authentication requirement, Example.

Swagger should be available in development.

44. TESTING

Minimum backend tests:

JUnit 5 and Mockito.

Test:

Assessment calculation, Skill score calculation, Gap calculation, Readiness calculation, Roadmap generation, Authorization, CV validation.

Integration testing:

Authentication, Database, Assessment flow, CV upload, AI service, Report generation.

45. CI/CD

Use GitHub Actions.

Pipeline:

Git Push → Build → Unit Tests → Integration Tests → Code Quality → Docker Build → Deploy

AWS deployment:

GitHub → GitHub Actions → Docker → ECR → ECS/Fargate

46. DOCKER

Backend should have Dockerfile and docker-compose.yml.

Local development: Spring Boot + PostgreSQL can run through Docker Compose.

47. ENVIRONMENT CONFIGURATION

Maintain:

application-dev.yml

application-test.yml

application-prod.yml

Environment variables:

DB_URL

DB_USERNAME

DB_PASSWORD

JWT_SECRET

AWS_REGION

AWS_S3_BUCKET

AWS_ACCESS_KEY

AWS_SECRET_KEY

AI_API_KEY

Production secrets must not be stored in source code.

48. AI COST CONTROL

Implement:

Token limits, Request limits, Response length limits, Caching where appropriate, Conversation limits, Admin AI usage dashboard.

Do not send unnecessary student data to the LLM. Send structured relevant data only.

Example:

```
{
  "targetCareer": "DevOps Engineer",
  "skillGaps": [
    {
      "skill": "Docker",
      "score": 55,
```

```
"required": 80
```

```
}  
]  
}
```

49. AI MODEL ABSTRACTION

Create AIProvider interface.

Example:

OpenAIProvider

OtherLLMProvider

FutureProvider

The rest of CareerOS should not depend directly on one provider.

50. AI AGENT ARCHITECTURE

Initial:

Career AI Assistant → Tool Calling → CareerOS Services

Future:

Career Coach with Skill Agent, CV Agent, Project Agent, Interview Agent and Roadmap Agent.

Do NOT implement multiple autonomous agents in MVP.

51. OBSERVABILITY

AWS CloudWatch:

Application logs, Error logs, CPU, Memory, API latency, ECS health.

Future:

OpenTelemetry, Grafana, Prometheus

52. PRODUCT UI

Design language:

Premium • Clean • Corporate • Student Friendly

Primary brand direction:

White, Orange, Yellow, Green, Dark neutral text

Avoid blue as the primary visual theme.

Dashboard should be simple rather than overloaded.

53. IMPORTANT UI SCREENS

Public:

Landing Page, Login, Registration, About CareerOS

Student:

Dashboard, Profile, Career Selection, Assessment, Assessment Result, Skill Gap, Roadmap, Projects, CV Analysis, LinkedIn Analysis, AI Assistant, Progress, Career Readiness Report, Settings

Admin:

Dashboard, Students, Careers, Skills, Questions, Assessments, Roadmaps, Projects, Reports, AI Configuration, Analytics, Settings

54. LANDING PAGE

Headline:

Know Where You Stand. Know What To Do Next.

Subheadline:

CODEIT CareerOS analyzes your skills, career gaps, CV and professional profile to create a personalized career action plan.

CTA:

Start Career Assessment

Secondary:

Explore CareerOS

55. MULTI-TENANT ARCHITECTURE

The first version does not need full college functionality.

Database architecture should be prepared for tenant_id.

Possible future tenants:

CODEIT, College A, College B, College C, Employer

Do not unnecessarily complicate MVP. Implement multi-tenancy properly when college deployment begins.

56. FUTURE COLLEGE PORTAL

College/TPO dashboard:

Total Students, Registered, Assessed, Career-wise distribution, Skill gaps, Projects, OJT, Internships, Readiness indicators.

Example:

College: ABC Engineering College

Students: 1,200

Assessed: 840

Top Career: Software Developer

Top Skill Gap: Cloud

Internship: 220

OJT: 145

57. FUTURE EMPLOYER PORTAL

Employer should eventually be able to search candidates using:

Career, Skills, Projects, Experience, OJT, Internship, Assessment, Availability, Location.

Only student-authorized information should be visible.

58. FUTURE CODEIT ECOSYSTEM INTEGRATION

CODEIT Academy → CareerOS → Project Hub → OJT → Internship → Resume / LinkedIn → Interview → Recruitment

This allows CODEIT to operate as an integrated student-to-employment ecosystem.

59. DEVELOPMENT PHASES

PHASE 1 – MVP:

Authentication, Student Profile, Career Master, Skills, Assessment, Assessment Result, Career Gap, Readiness Score, Roadmap, CV Analysis, LinkedIn Analysis, Project Recommendation, AI Assistant, PDF Report, Admin Dashboard.

PHASE 2:

Progress Tracking, Mock Interviews, AI Project Mentor, Daily Career Tasks, Advanced Assessments, College Dashboard, OJT Management, Internship Management.

PHASE 3:

Employer Portal, Recruitment, Job Matching, Campus Hiring, College Multi-Tenant, Employer Search, Candidate Shortlisting.

60. DEVELOPMENT PRIORITY

Sprint 1:

Project setup, Database, Authentication, User roles, Student profile

Sprint 2:

Career master, Skill master, Competency framework, Assessment engine

Sprint 3:

Assessment results, Career gap engine, Readiness score, Dashboard

Sprint 4:

Roadmap, Project recommendations, Progress tracking

Sprint 5:

CV upload, CV parser, CV AI analysis

Sprint 6:

LinkedIn analysis, Career Readiness Report

Sprint 7:

Spring AI, AI Career Assistant, Tool calling, AI chat history

Sprint 8:

Testing, Security, Docker, AWS deployment, Monitoring, Production release

61. DEFINITION OF DONE

A module is complete only when:

Backend API completed

Database completed

Frontend completed

Validation completed

Authorization completed

Unit tests completed

Integration test completed

Swagger documentation completed

Error handling completed

Logging completed

Do not consider a feature complete merely because the UI is working.

62. GIT BRANCHING

Recommended:

main

develop

feature/auth

feature/assessment

feature/career-gap

feature/cv-analysis

feature/ai-assistant

Pull request required before merging to develop or main.

63. IMPORTANT DEVELOPMENT RULES

- 1. Do not hard-code business rules.**
- 2. Do not put AI logic directly inside controllers.**
- 3. Do not allow AI direct database access.**
- 4. Do not expose database entities directly.**
- 5. Use DTOs.**
- 6. Use service layer.**
- 7. All important calculations must be deterministic and testable.**
- 8. AI should explain and personalize, not become the source of truth for verified student data.**
- 9. Keep AI provider replaceable.**

10. Design APIs for future mobile applications.

64. RECOMMENDED PACKAGE ARCHITECTURE

com.codeit.careeros

- auth
 - ■■■ controller
 - ■■■ service
 - ■■■ repository
 - ■■■ entity
 - ■■■ dto
- student
- career
- skill
- assessment
- gap
- readiness
- roadmap
- project
- cv
- linkedin
- progress
- report
- ai
 - ■■■ controller
 - ■■■ service
 - ■■■ tools
 - ■■■ prompt
 - ■■■ model
- admin
- security
- config
- exception
- common

65. INITIAL DELIVERABLES FROM DEVELOPER

1. GitHub repository
2. Database ER diagram
3. API documentation
4. Swagger
5. Backend source code
6. Frontend source code

7. Database migration scripts
8. Docker configuration
9. Unit tests
10. Integration tests
11. AWS deployment documentation
12. Environment configuration documentation
13. AI integration documentation
14. Admin credentials setup procedure
15. Production deployment procedure

66. REQUIRED DOCUMENTATION BEFORE CODING

Document 1: System Architecture

Document 2: Database ER Diagram

Document 3: API Specification

Document 4: UI Wireframes

Document 5: AI Architecture

Document 6: Deployment Architecture

After approval, coding should begin.

67. CORE PRODUCT PRINCIPLE

CODEIT CareerOS should not simply tell a student: "You need to learn Docker."

It should tell the student:

"Your target is DevOps Engineer. Your current demonstrated Docker competency is 55%, while the configured target competency is 80%. This is a priority gap. Complete the recommended Docker project, then retake the assessment."

That is the difference between a normal learning platform and CareerOS.

68. FINAL SYSTEM ARCHITECTURE

CODEIT CareerOS

- Student Platform
- Admin Platform
- Next.js Frontend
- REST APIs

- Spring Boot Java 21
- ■■■ Assessment Engine
- ■■■ Career Engine
- ■■■ CV Analysis
- ■■■ LinkedIn Analysis
- ■■■ AI Assistant
- Business Services
- PostgreSQL Database
- S3 Documents/Reports
- AWS Services
- AI Provider
- AWS ECS / Fargate + CloudWatch

69. FUTURE CODEIT ECOSYSTEM

CODEIT

- CODEIT CareerOS
- Academy
- Projects
- OJT
- Internship
- Recruitment
- Student Profile → Employer Network → Placement

The objective is to create a student career operating system, not just another training website.

70. FINAL DEVELOPER INSTRUCTION

Build CODEIT CareerOS as an API-first, Java/Spring Boot, AI-ready platform.

First priority:

Student → Career → Assessment → Skill Gap → Readiness → Roadmap → AI Guidance

Do not begin by developing every future module. Build a clean, secure and scalable MVP first.

Every major business rule must be configurable.

Every AI capability must operate through controlled tools and CareerOS services.

The platform must be designed so future modules such as College Dashboard → OJT → Internship → Employer → Recruitment → Job Matching can be added without rebuilding the core system.

Primary technology:

Java 21 + Spring Boot + PostgreSQL + React/Next.js + Spring AI + AWS.

Primary product:

CODEIT CareerOS.

Primary outcome:

Help students understand their current career readiness, identify their gaps and follow a measurable path toward industry readiness.